

Praktikum

BMC4: Embedded Systems und Feldbusse, Teil Embedded Systems

V1: Embedded Systems MSP430 - Polling- versus Interruptbetrieb

Prof. Dr.-Ing. Ludwig Eckert
ludwig.eckert@thws.de

Labor Prozessdatenverarbeitung und eingebettete Systeme,
Raum 1.1.06

Inhaltsverzeichnis

- 1. Download der Praktikumsunterlagen / Veranstaltungsort**
2. Ziele und Aufgabenbeschreibung des Versuchs V1
3. Rekapitulation Binäre Operatoren
4. Überblick Mikrocontroller TI MSP430 EVAL Board
5. Der digitale Ein- und Ausgabeport P1
6. Übung LED Blinken
7. Die Low Power Modi des Mikrocontrollers TI MSP430

Download der Praktikumsunterlagen:

[V1: Embedded Systems MSP430 - Polling- versus Interruptbetrieb](#)

<https://elearning.thws.de/course/view.php?id=28767>

Veranstaltungsort:

Labor Prozessdatenverarbeitung u. eingebettete Systeme
im Raum 1.1.06

Inhaltsverzeichnis

1. Download der Praktikumsunterlagen / Veranstaltungsort
- 2. Ziele und Aufgabenbeschreibung des Versuchs V1**
3. Rekapitulation Binäre Operatoren
4. Überblick Mikrocontroller TI MSP430 EVAL Board
5. Die serielle Schnittstelle
6. Der digitale Ein- und Ausgabeport P1
7. Übung LED Blinken
8. Die Low Power Modi des Mikrocontrollers TI MSP430

TEILVERSUCH 1: POLLING EINES TASTERS ZUR ANSTEUERUNG DER LED

Beeinflussung des Programmablaufs unter Verwendung der digitalen Input/Output-Portschnittstelle (GPIO-Unit³).

Ziele und Aufgabenbeschreibung

Beim nächsten Programm „Versuch1.c“ soll eine auf dem Board vorhandene LED kurz blinken, nachdem der Taster B5 betätigt wurde. Die LED quittiert so das Betätigen des Tasters.

Die Abfrage des Tasters B5 erfolgt durch zyklisches Einlesen des digitalen Pegels am Portpin P2.1, was als Polling bezeichnet wird.

Voraussetzung für diesen Versuch ist ein Verständnis für die Ein- und Ausgabe über die parallelen Ports P1 und Port 2 des MSP430F449 Mikrocontrollers, ebenso wie eine Strommessung für die Tabelle unter 5.2 soll durchgeführt werden kann.

TEILVERSUCH 2: INTERRUPT-BETRIEB

Interrupt-Verarbeitung des Mikrocontroller MSP430F449.

Ziele und Aufgabenbeschreibung

Ziel des Versuches ist das Kennenlernen der Interrupt-Verarbeitung des Mikrocontrollers MSP430F449.

Im Teilversuch 2 soll nun auf das energieintensive Polling verzichtet werden und eine ereignisgesteuerte, d. h. interruptorientierte Implementierung des Anwendungsfalles angestrebt werden.

Wird der Taster B5 betätigt, so wird intern ein Interrupt (hier Portinterrupt) ausgelöst und die zugeordnete Interruptservicefunktion `ISR_Port2()` gestartet. In der `ISR_Port2()` wird die LED wechselweise ein- bzw. ausgeschaltet.

Inhaltsverzeichnis

1. Download der Praktikumsunterlagen / Veranstaltungsort
2. Ziele und Aufgabenbeschreibung des Versuchs V1
- 3. Rekapitulation Binäre Operatoren**
4. Überblick Mikrocontroller TI MSP430 EVAL Board
5. Der digitale Ein- und Ausgabeport P1
6. Übung LED Blinken
7. Die Low Power Modi des Mikrocontrollers TI MSP430

Rekapitulation Binäre gemäß ISO Standard C



Technische Hochschule
Würzburg Schweinfurt

Binäre Operatoren - Übersicht

Begriffsdefinitionen

Einsatz von symbolischen Konstanten

Definition von Bitgruppen

Binäre Operatoren OR, AND, XOR

Bsp.: Automatisierung einer Waschmaschine

Selektives Setzen von Bits mittels OR

Selektives Rücksetzen von Bits mittels AND

Selektives Toggeln einzelner Bitpositionen mittels XOR

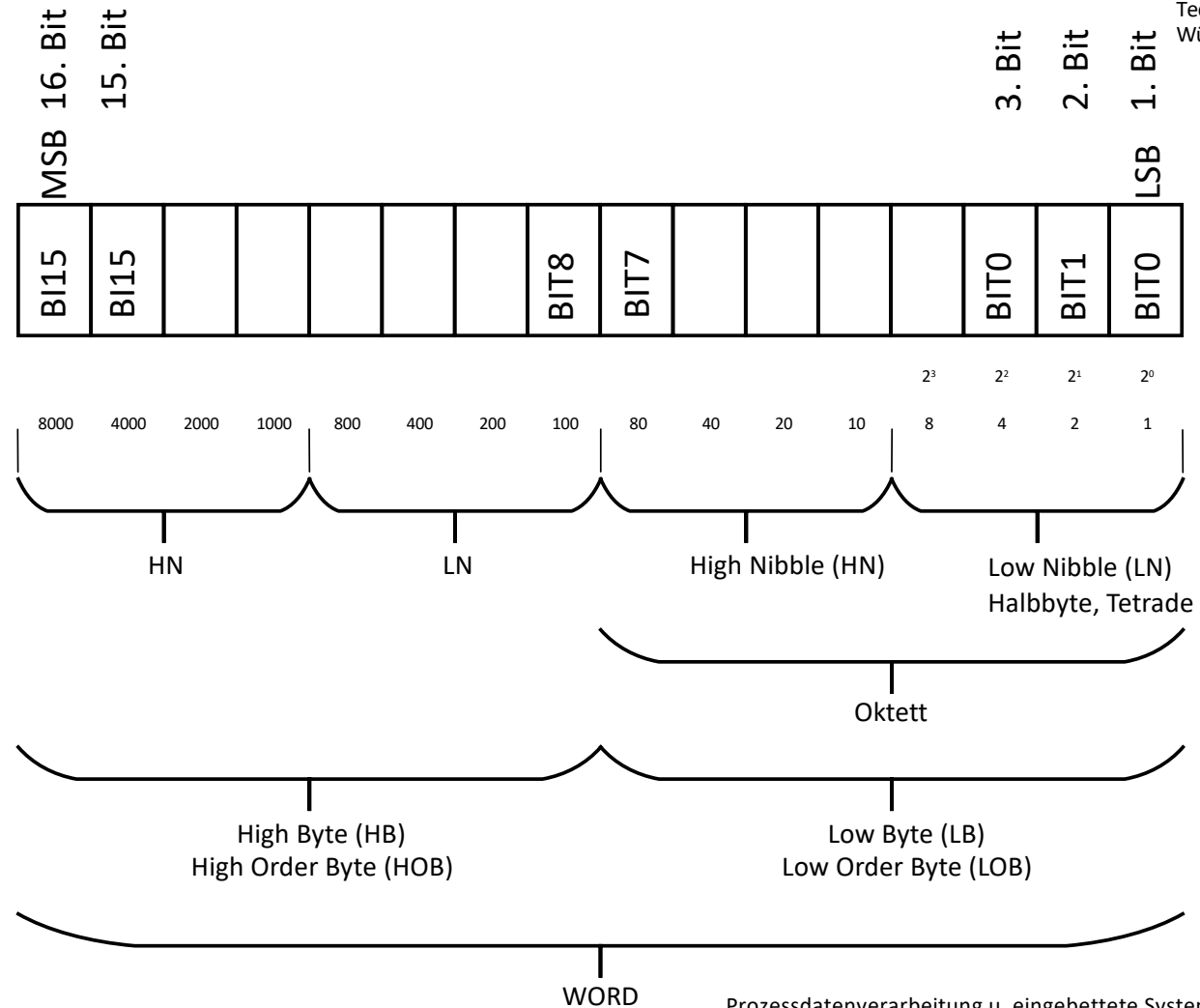
Übungen zu Bitoperationen

Übung - Grundlagen Binäre Operationen

Übung - Registermanipulationen, Abfrage Statusregister, PORTkonfiguration

Übung - Statusabfrage einer seriellen UART-Schnittstelle

Allgemeine Begriffsdefinitionen



Legende:

MSB - Most Significant Bit

LSB - Least Significant Bit

Binäre Operationen gemäß ISO Standard C

OPERATOR	ANWENDUNG	BEDEUTUNG
& AND	$n3 = n2 \& n1;$ $n2 = n2 \& n1;$ oder $n2 \&= n1;$	$n3$ erhält das Ergebnis von $n1$ AND $n2$; $n2$ erhält das Ergebnis von $n1$ AND $n2$;
 OR	$n3 = n2 n1;$ $n2 = n2 n1;$ oder $n2 = n1;$	$n3$ erhält das Ergebnis von $n1$ OR $n2$; $n2$ erhält das Ergebnis von $n1$ OR $n2$;
^ EXCLUSIV-OR (XOR)	$n3 = n2 ^ n1;$ $n2 = n2 ^ n1;$ oder $n2 ^= n1;$	$n3$ erhält das Ergebnis von $n1$ XOR $n2$; $n2$ erhält das Ergebnis von $n1$ XOR $n2$;
~ COMPLEMENT	$n3 = n3 \sim n1;$ $n1 = n1 \sim n1;$ oder $n1 \sim= n1;$	$n3$ erhält das Ergebnis des KOMPLEMENTs von $n1$; $n1$ erhält das Ergebnis des KOMPLEMENTs von $n1$;
<< SHIFT LEFT shl	$n3 = n1 << x;$ $n1 = n1 << x;$ oder $n1 <<= x;$	$n3$ erhält das Ergebnis von $n1$ mit x -mal SHIFT LINKS $n1$ erhält das Ergebnis von $n1$ mit x -mal SHIFT LINKS
>> SHIFT RIGHT shr	$n3 = n1 >> x;$ $n1 = n1 >> x;$ oder $n1 >>= x;$	$n3$ erhält das Ergebnis von $n1$ mit x -mal SHIFT RECHTS $n1$ erhält das Ergebnis von $n1$ mit x -mal SHIFT RECHTS

Verwendung von symbolischen Konstanten zur Erhöhung der Lesbarkeit von Programmen.

```
#define BIT0 0x01
#define BIT1 0x02
#define BIT2 0x04
#define BIT3 0x08
```

üblicherweise Großbuchstaben

P1OUT =

BIT7	BIT6	BIT5	BIT4	BIT3	BIT2	BIT1	BIT0
0	0	0	0	0	1	0	1

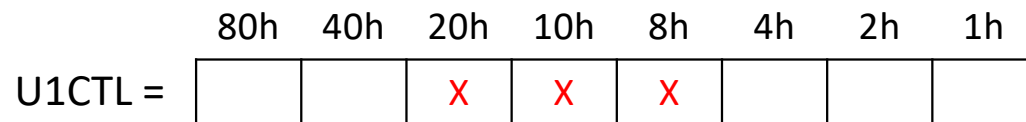
P1OUT = 0x05;

alternativ:

P1OUT = BIT0 | BIT2; [Headerfile msp430x44x.h zum Mikrocontroller MSP430F449](#)

Anm.: Symbolische Konstanten werden durch den Präprozessor vor dem Compilerlauf durch die zugehörigen define-Werte ersetzt.

Verwendung von symbolischen Konstanten zur Erhöhung der Lesbarkeit von Programmen: Definition von Bitgruppen



Bitgruppe

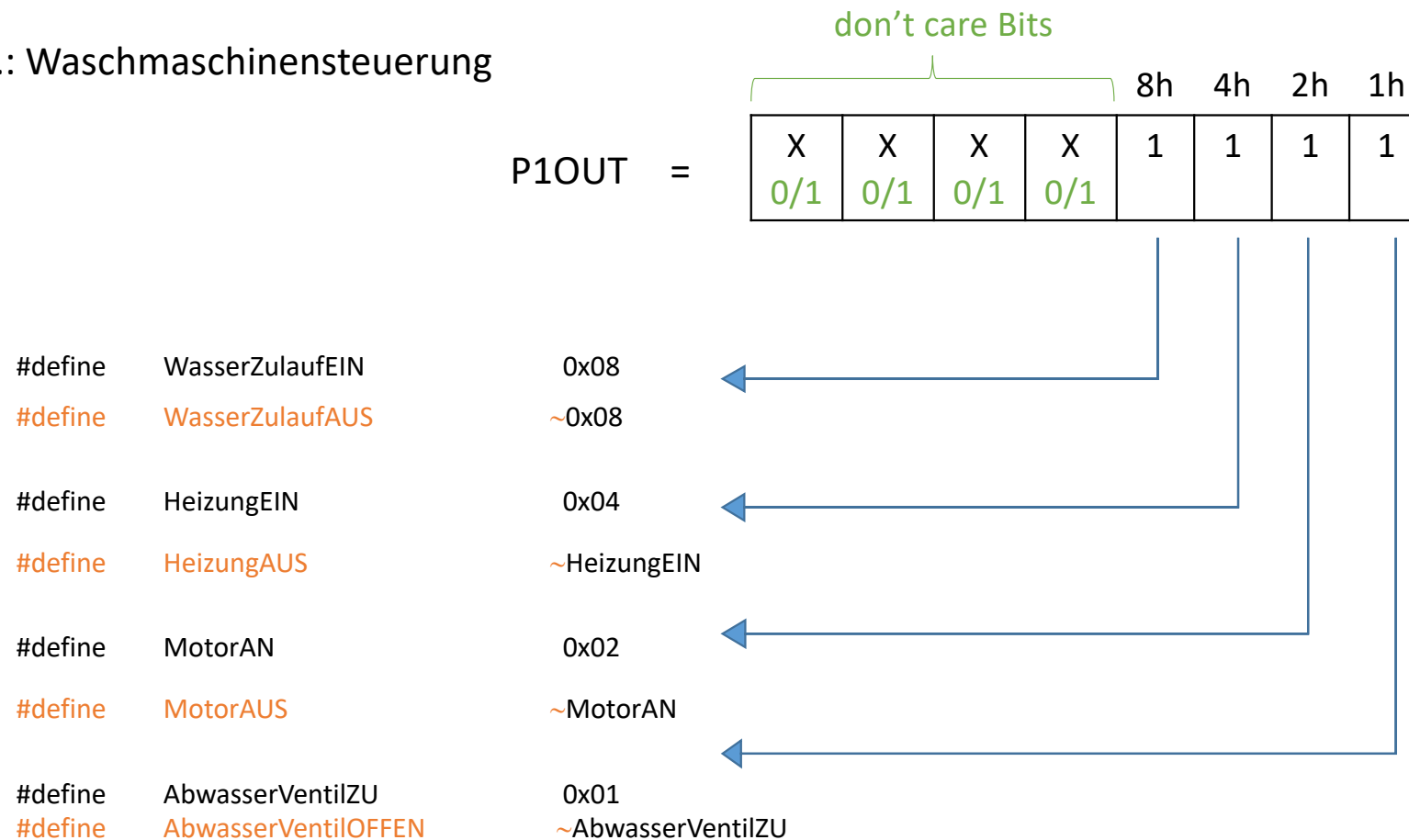
0	0	0	0	2400 Bd
1	0	0	1	4800 Bd
2	0	1	0	9600 Bd
3	0	1	1	19200 Bd
4	1	0	0	38400 Bd
5	1	0	1	76800 Bd
6	1	1	0	153600 Bd
7	1	1	1	307200 Bd

==>

```
# define  Basisadresse  0x08
# define  BD_2400        0 x Basisadresse
# define  BD_4800        1 x Basisadresse
# define  BD_9600        2 x Basisadresse
# define  BD_19200       3 x Basisadresse
:
oder alternativ:
# define  BD_2400        0 x 0x08
# define  BD_4800        1 x 0x08
# define  BD_9600        2 x 0x08
# define  BD_19200       3 x 0x08
:
```

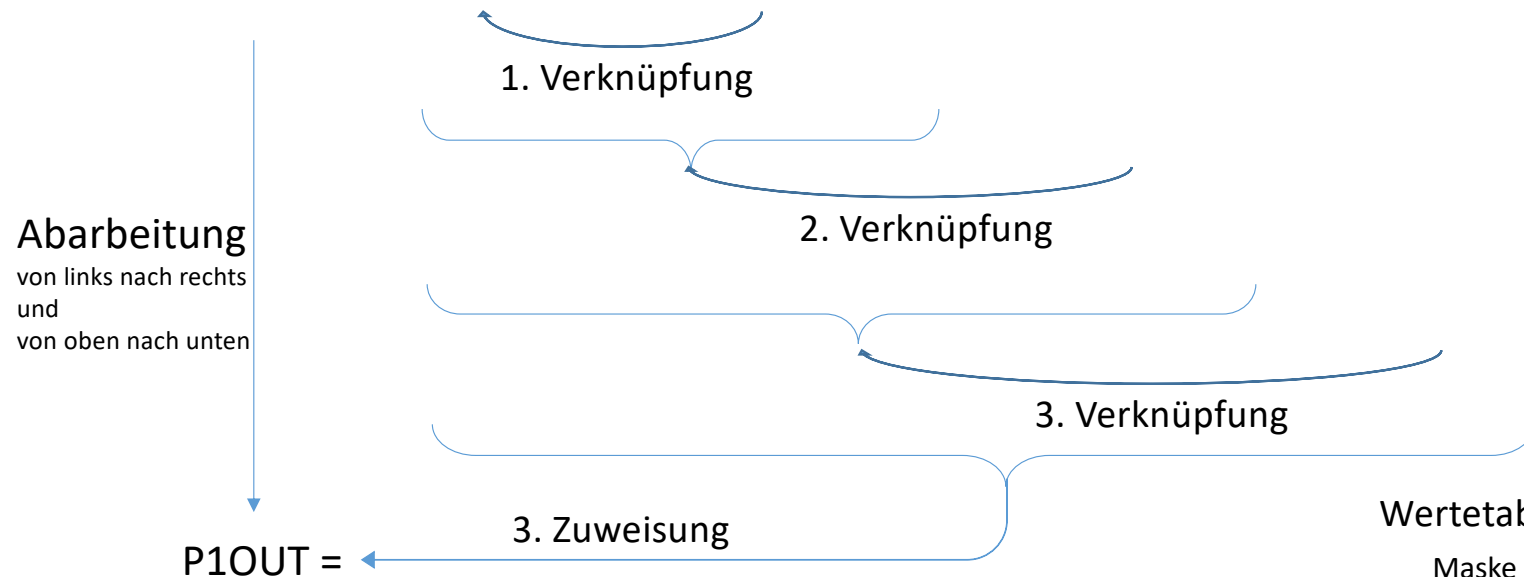
Weitere Verbesserung der Lesbarkeit durch anwendungsbezogene define-Anweisungen

Bsp.: Waschmaschinensteuerung



Selektives Setzen einzelner Bits mittels Bitoperator OR

$P1OUT = P1OUT \mid \text{WasserzulaufEIN} \mid \text{AbwasserVentilZU} \mid \text{HeizungEIN};$



vorher $P1OUT =$

X	X	X	X	0	0	0	0
---	---	---	---	---	---	---	---

nachher $P1OUT =$

X	X	X	X	1	1	0	1
---	---	---	---	---	---	---	---

Wertetabelle

Maske		
A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Selektives Rücksetzen einzelner Bits mittels Bitoperator *AND*

vorher P1OUT = 0 0 0 0 1 1 0 1
 AND, & 1 1 1 1 0 1 1 1
 nachher P1OUT = 0 0 0 0 0 1 0 1

Wertetabelle
Maske

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Verhalten:

Eine 0 in der Bitmaske maskiert (blendet) die entsprechende Bitposition aus

P1OUT = P1OUT & ~WasserZulaufEIN; // ~ (Tilde) $\triangleq$ 1er Komplement

oder

P1OUT = P1OUT & WasserZulaufAUS;

bzw.

P1OUT = P1OUT & HeizungAUS | MotorAN;

Selektives Toggeln einzelner Bitpositionen mittels Bitoperator *XOR*

$$\begin{array}{rcl}
 \text{P1OUT} & = & \begin{array}{|c|c|c|c|c|c|c|c|} \hline X & X & X & X & 1 & X & X & 1 \\ \hline \end{array} \\
 \text{XOR, } ^ & & \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline \end{array} \quad \text{Bitmaske} \\
 & & \hline
 & & \begin{array}{|c|c|c|c|c|c|c|c|} \hline X & X & X & X & 0 & X & X & 1 \\ \hline \end{array}
 \end{array}$$

`P1OUT = P1OUT ^ WasserZulaufEIN; // Zustandwechsel`

Bsp.:

$$\begin{array}{rcl}
 \text{P1OUT} & = & \begin{array}{|c|c|c|c|c|c|c|c|} \hline X & X & X & X & 1 & 0 & 0 & 1 \\ \hline \end{array} \\
 \text{XOR} & & \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline \end{array} \quad \text{Bitmaske} \\
 & & \hline
 & & \begin{array}{|c|c|c|c|c|c|c|c|} \hline \bar{X} & X & X & X & 0 & 0 & 0 & 1 \\ \hline \end{array}
 \end{array}$$

Wertetabelle
Maske

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Verhalten: Eine 1 in der Bitmaske invertiert (toggelt) die entsprechende Bitposition in P1OUT, eine 0 übernimmt den vorherigen Wert.

Übung 1

Binäre Operatoren

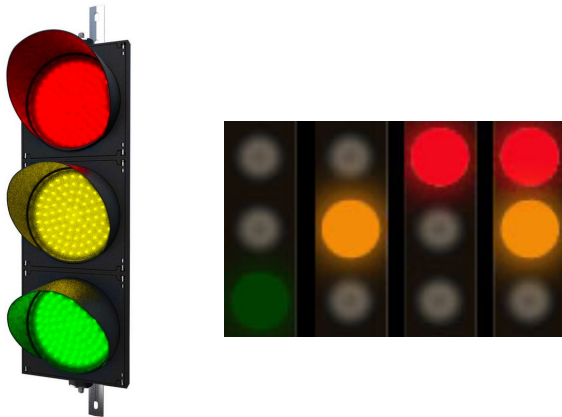
[Download Übung 1: Binäre Operatoren](#)

Übung 2

Registermanipulationen, Abfrage Statusregister, PORT-Konfiguration

[Download Übung 2- Registermanipulationen, Abfrage Statusregister, PORTkonfiguration](#)

Übung 3: Implementierung Sie in C oder einer Pseudosprache nachstehende Ampelsteuerung

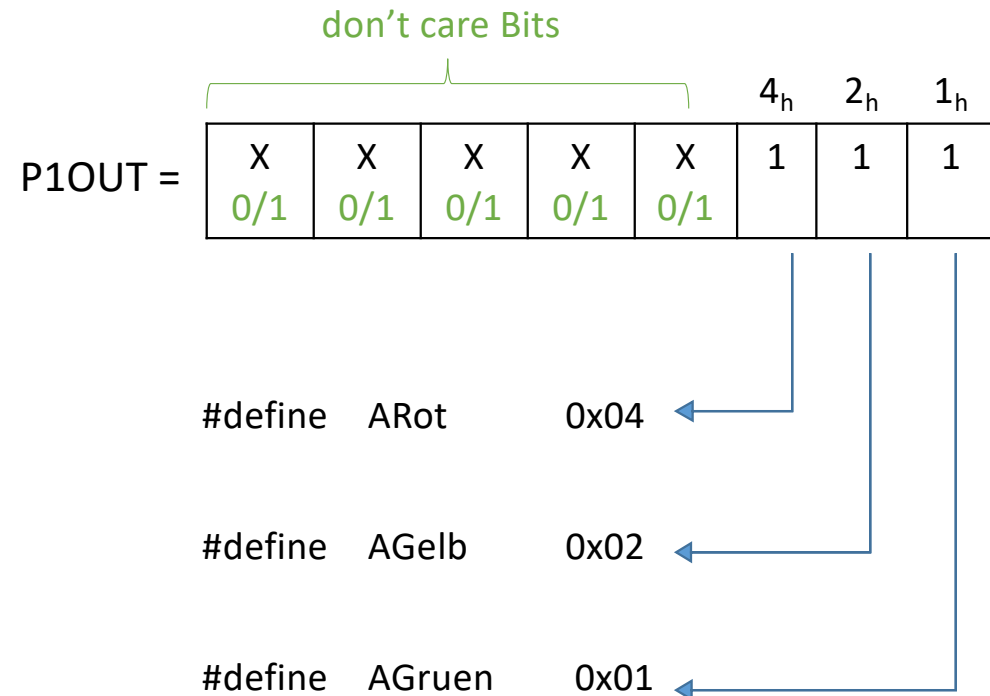


Die vier Phasen sind grün, gelb, rot und rot-gelb:

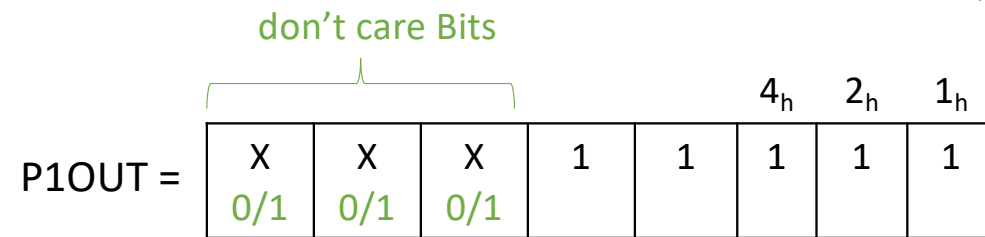
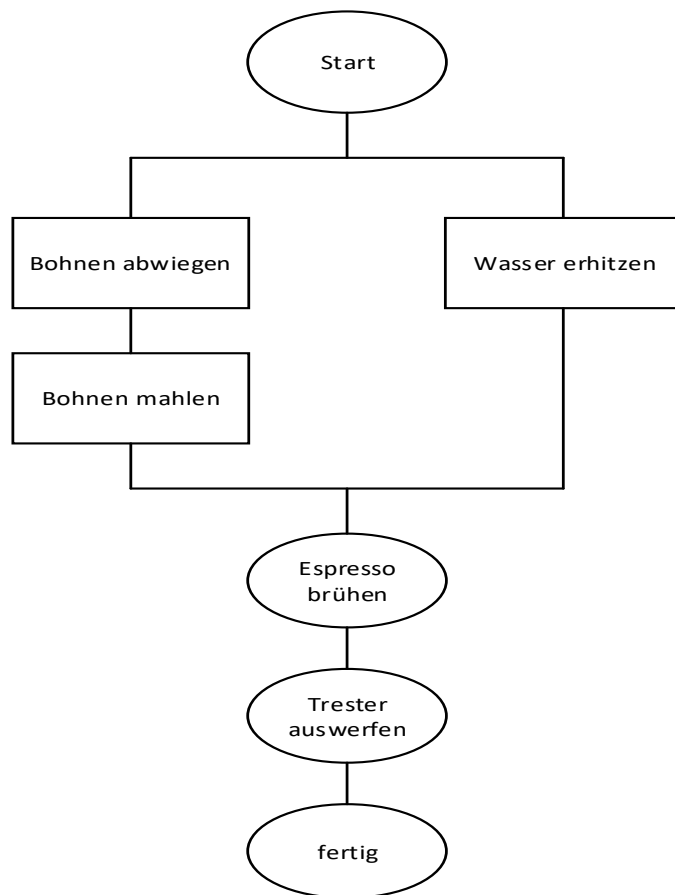
- **Grün** bedeutet: "Der Verkehr ist freigegeben"
- **Gelb** ordnet an: "Vor der Kreuzung auf das nächste Zeichen warten"
- **Rot** ordnet an: "Halt vor der Kreuzung"
- **Rot-gelb** ordnet an „auf die Weiterfahrt vorbereiten“

Alle vier Ampelphasen laufen nacheinander ab und wiederholen sich endlos.

Für die Realisierung des zeitlichen Abstandes zwischen den Ampelphasen und die Anzeigedauer der jeweiligen Ampelphase steht die Funktion *delay(unsigned int sec)* zur Verfügung.



Übung 4: Implementierung einer Steuerung für einen Espressoautomaten



#define _____ 0x04

#define _____ 0x02

#define _____ 0x01

Inhaltsverzeichnis

1. Download der Praktikumsunterlagen / Veranstaltungsort
2. Ziele und Aufgabenbeschreibung des Versuchs V1
3. Rekapitulation Binäre Operatoren
- 4. Überblick Mikrocontroller TI MSP430 EVAL Board**
5. Der digitale Ein- und Ausgabeport P1
6. Übung LED Blinken
7. Die Low Power Modi des Mikrocontrollers TI MSP430

Versuchsanordnung Versuch V1 und V2

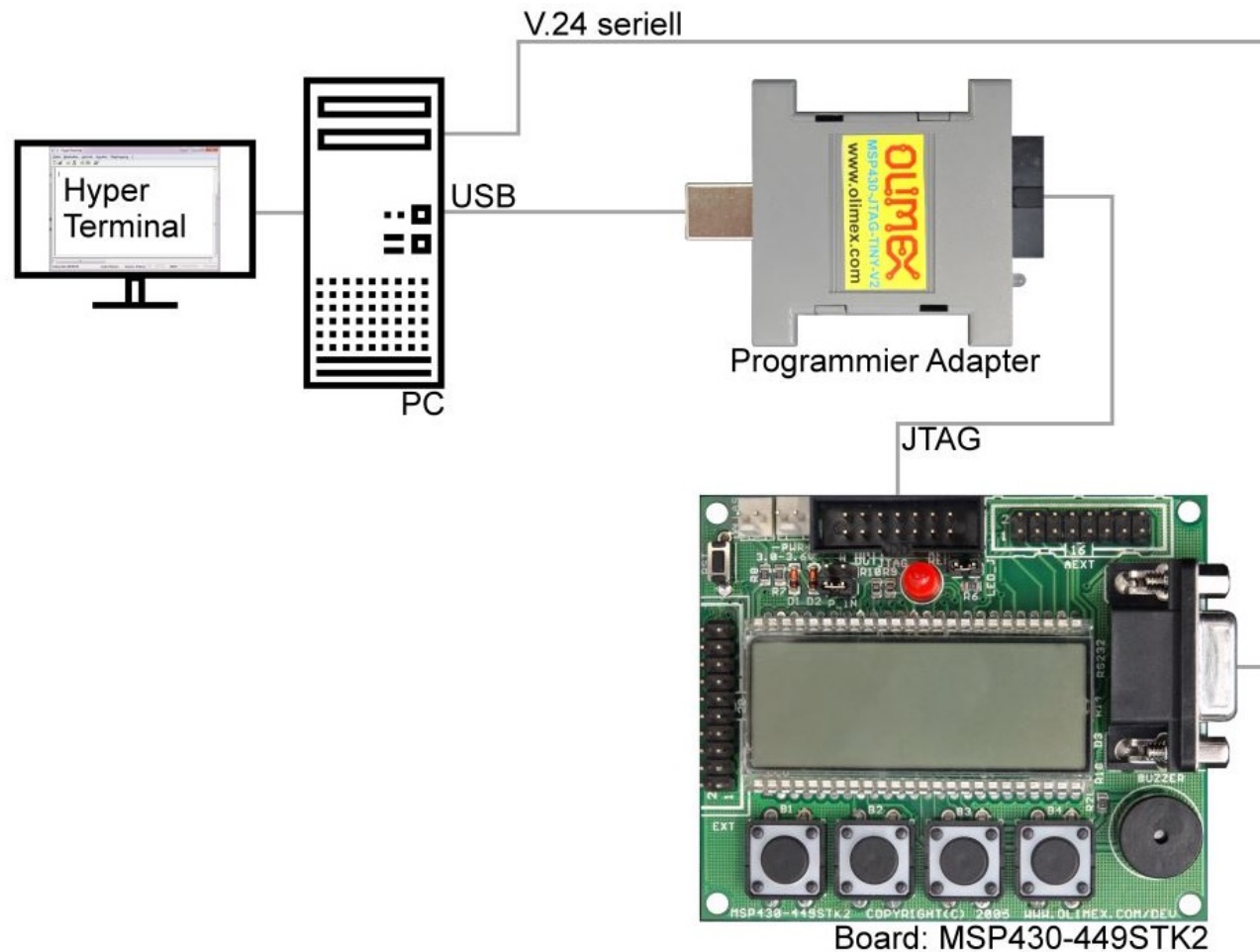
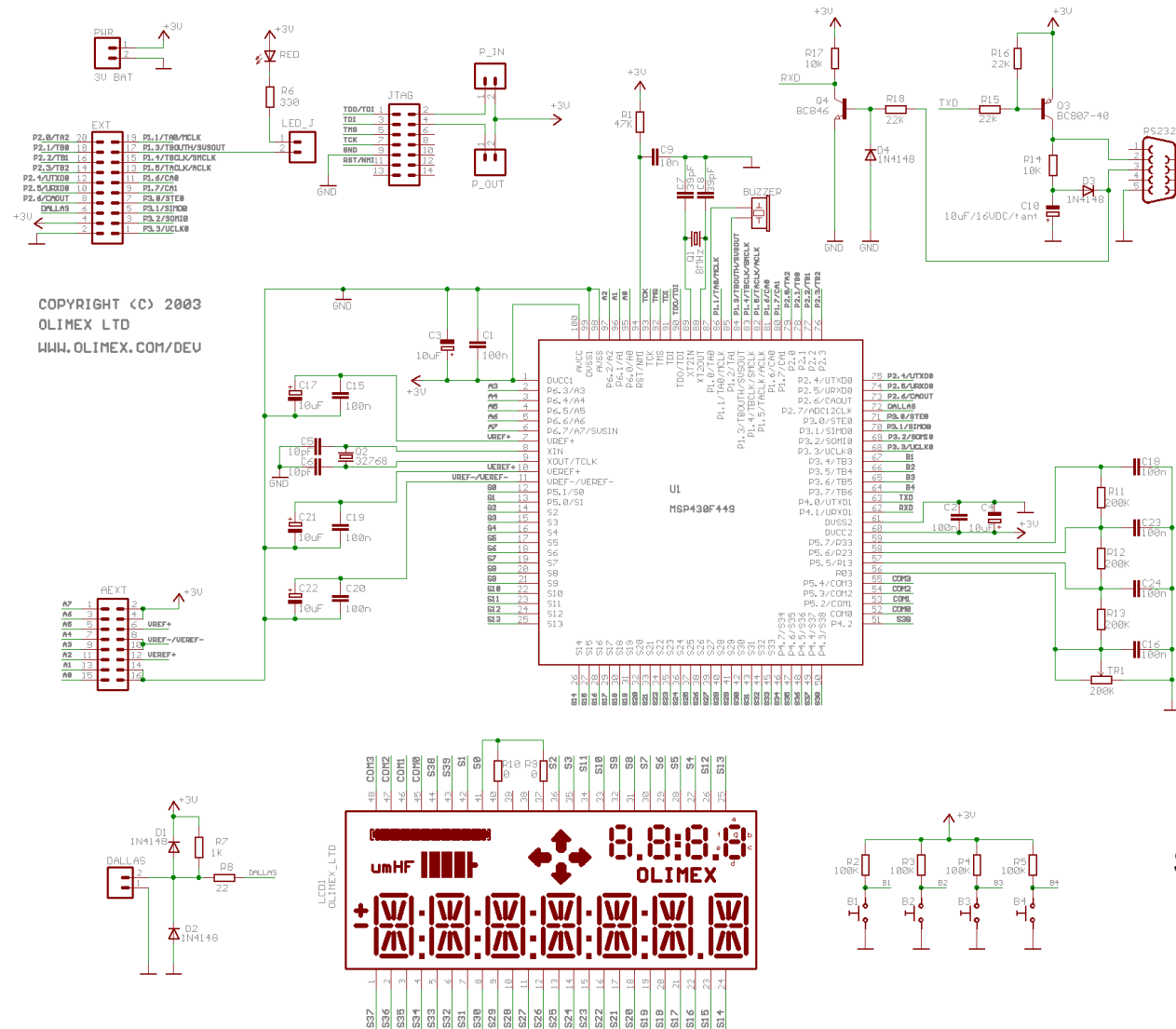


Bild 7: Anschluss des MSP430 Boards mit Spannungsversorgung über die JTAG-Schnittstelle

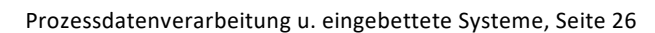
Versuchsanordnung Versuch V1 / V2



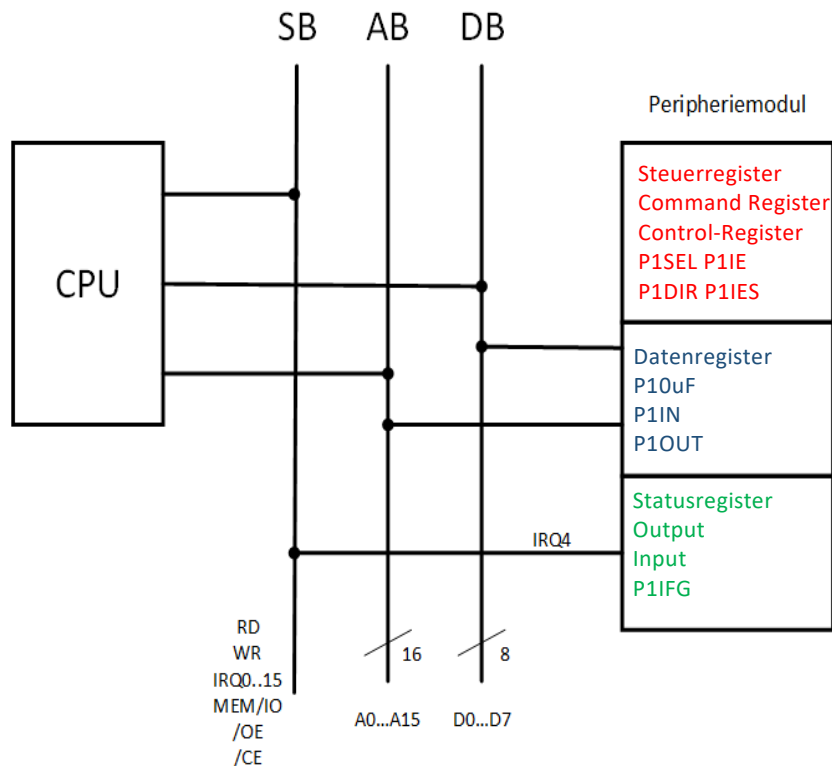
Technische Hochschule
Würzburg Schweinfurt



Schaltplan des OLIMEX MSP430 Eval Board v2



Interface zum Input/Output-Peripheriebaustein - Allgemein



Steuerregister: dienen der Festlegung der Betriebsart (Command-Register, Control, CTL, Konfigurationsregister).

PxDIR – Pin Direction

PxSEL – Pin Select

PxIE – P1 Interrupt Enable

PxIES – P1 Interrupt Edge Select

Datenregister: dienen dem Austausch von Daten zwischen der CPU und dem Peripheriemodul.

char temp = P1IN;

P2OUT= temp;

Statusregister: geben Auskunft über den Zustand des Peripheriemoduls, z. B. Informieren über das Eintreffen neuer Datenbytes oder geben Auskunft über evtl. aufgetretene Übertragungsfehler oder Signalwechsel während der Übertragung.

PxIFG – P1 Interrupt Flag

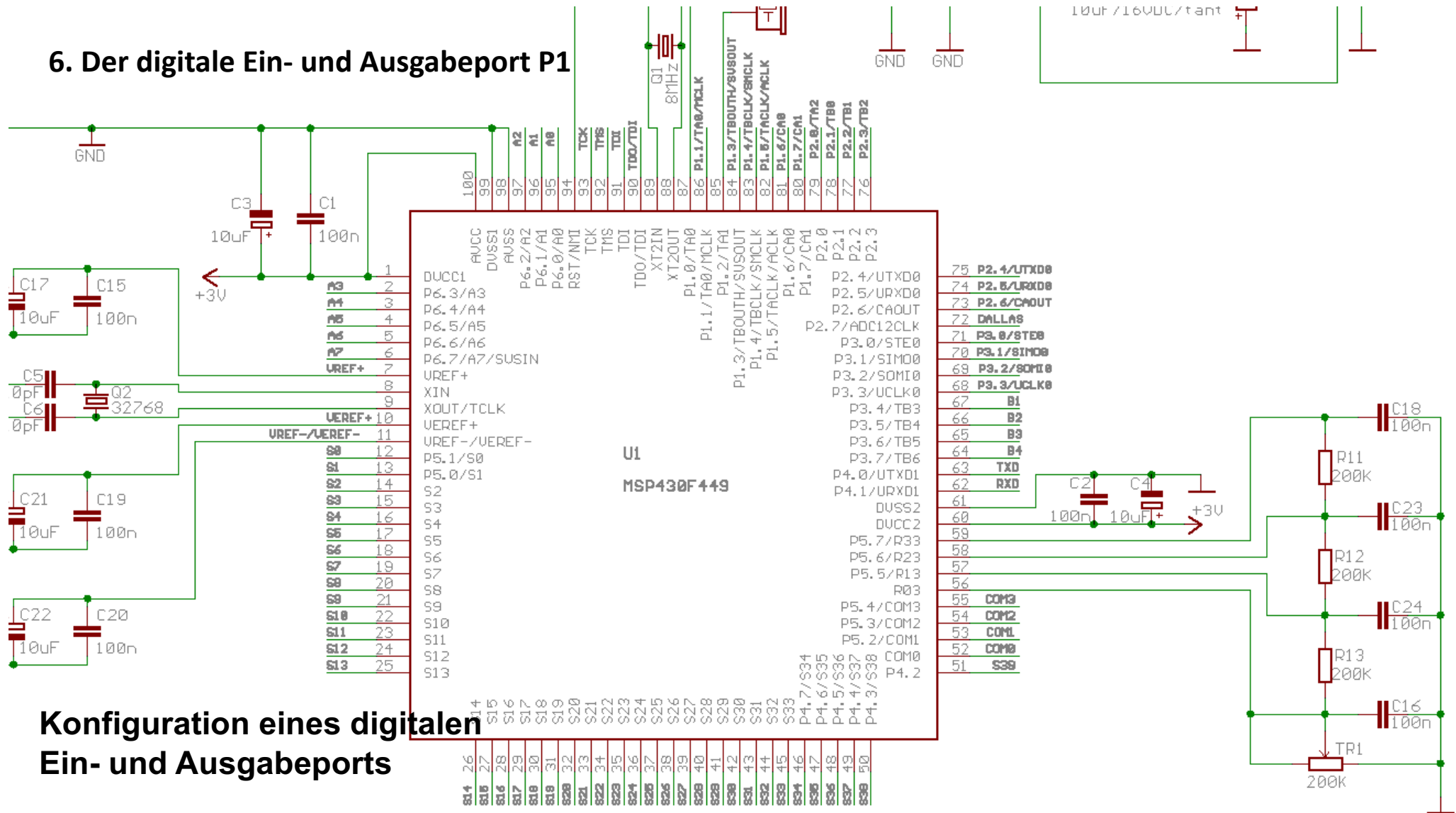
Inhaltsverzeichnis

1. Download der Praktikumsunterlagen / Veranstaltungsort
2. Ziele und Aufgabenbeschreibung des Versuchs V1
3. Rekapitulation Binäre Operatoren
4. Überblick Mikrocontroller TI MSP430 EVAL Board
5. Der digitale Ein- und Ausgabeport P1
6. Übung LED Blinken
7. Die Low Power Modi des Mikrocontrollers TI MSP430

Inhaltsverzeichnis

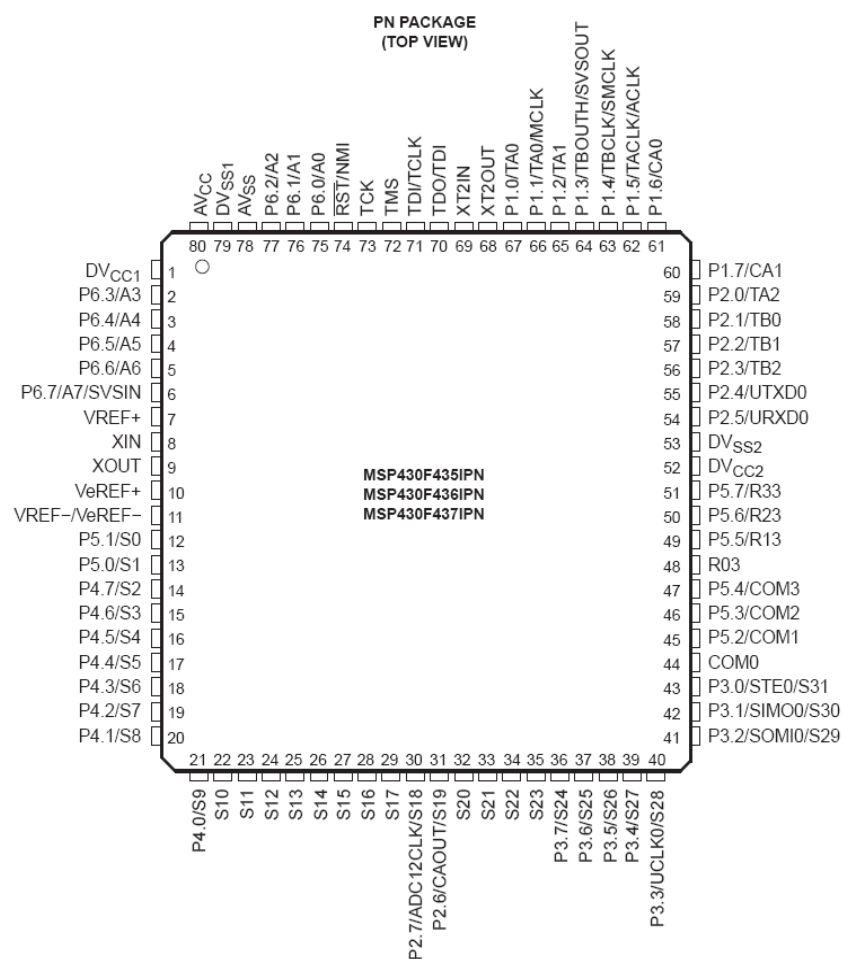
1. Download der Praktikumsunterlagen / Veranstaltungsort
2. Ziele und Aufgabenbeschreibung des Versuchs V1
3. Rekapitulation Binäre Operatoren
4. Überblick Mikrocontroller TI MSP430 EVAL Board
- 5. Der digitale Ein- und Ausgabeport P1**
6. Übung LED Blinken
7. Die Low Power Modi des Mikrocontrollers TI MSP430

6. Der digitale Ein- und Ausgabeport P1



Konfiguration eines digitalen Ein- und Ausgabeports

Merhrfachverwendung einzelner MCU-Pins



TERMINAL		I/O	DESCRIPTION
PN NAME	NO.		
P1.6/CA0	81	I/O	General-purpose digital I/O / Comparator_A input
P1.5/TACLK/ ACLK	82	I/O	General-purpose digital I/O / Timer_A, clock signal TACLK input / ACLK output (divided by 1, 2, 4, or 8)
P1.4/TBCLK/ SMCLK	83	I/O	General-purpose digital I/O / input clock TBCLK—Timer_B7 / submain system clock SMCLK output
P1.3/TBOutH/ SVSOut	84	I/O	General-purpose digital I/O / switch all PWM digital output ports to high impedance—Timer_B7 TB0 to TB6 / SVS: output of SVS comparator
P1.2/TA1	85	I/O	General-purpose digital I/O / Timer_A, Capture: CCI1A, compare: Out1 output
P1.1/TA0/MCLK	86	I/O	General-purpose digital I/O / Timer_A, Capture: CCI0B / MCLK output. Note: TA0 is only an input on this pin.
P1.0/TA0	87	I/O	General-purpose digital I/O / Timer_A, Capture: CCI0A input, compare: Out0 output
XT2OUT	88	O	Output terminal of crystal oscillator XT2
XT2IN	89	I	Input port for crystal oscillator XT2. Only standard crystals can be connected.
TDO/TDI	90	I/O	Test data output port. TDO/TDI data output or programming data input terminal
TDI	91	I	Test data input. TDI is used as a data input port. The device protection fuse is connected to TDI.
TMS	92	I	Test mode select. TMS is used as an input port for device programming and test.
TCK	93	I	Test clock. TCK is the clock input port for device programming and test.
RST/NMI	94	I	Reset input or nonmaskable interrupt input port
P6.0/A0	95	I/O	General-purpose digital I/O, analog input a0—12-bit ADC
P6.1/A1	96	I/O	General-purpose digital I/O, analog input a1—12-bit ADC
P6.2/A2	97	I/O	General-purpose digital I/O, analog input a2—12-bit ADC
AVSS	98		Negative terminal that supplies SVS, brownout, oscillator, FLL+, comparator_A, port 1, and LCD resistive divider circuitry.
DVSS1	99		Digital supply voltage, negative terminal. Supplies all digital parts, except those which are supplied via AVCC/AVSS.
AVCC	100		Positive terminal that supplies SVS, brownout, oscillator, FLL+, comparator_A, port 1, and LCD resistive divider circuitry; must not power up prior to DVCC1/DVCC2.

6. Der digitale Ein- und Ausgabeport P1

Control-, Daten- und Status-Register der digitalen Ein- und Ausgabeports

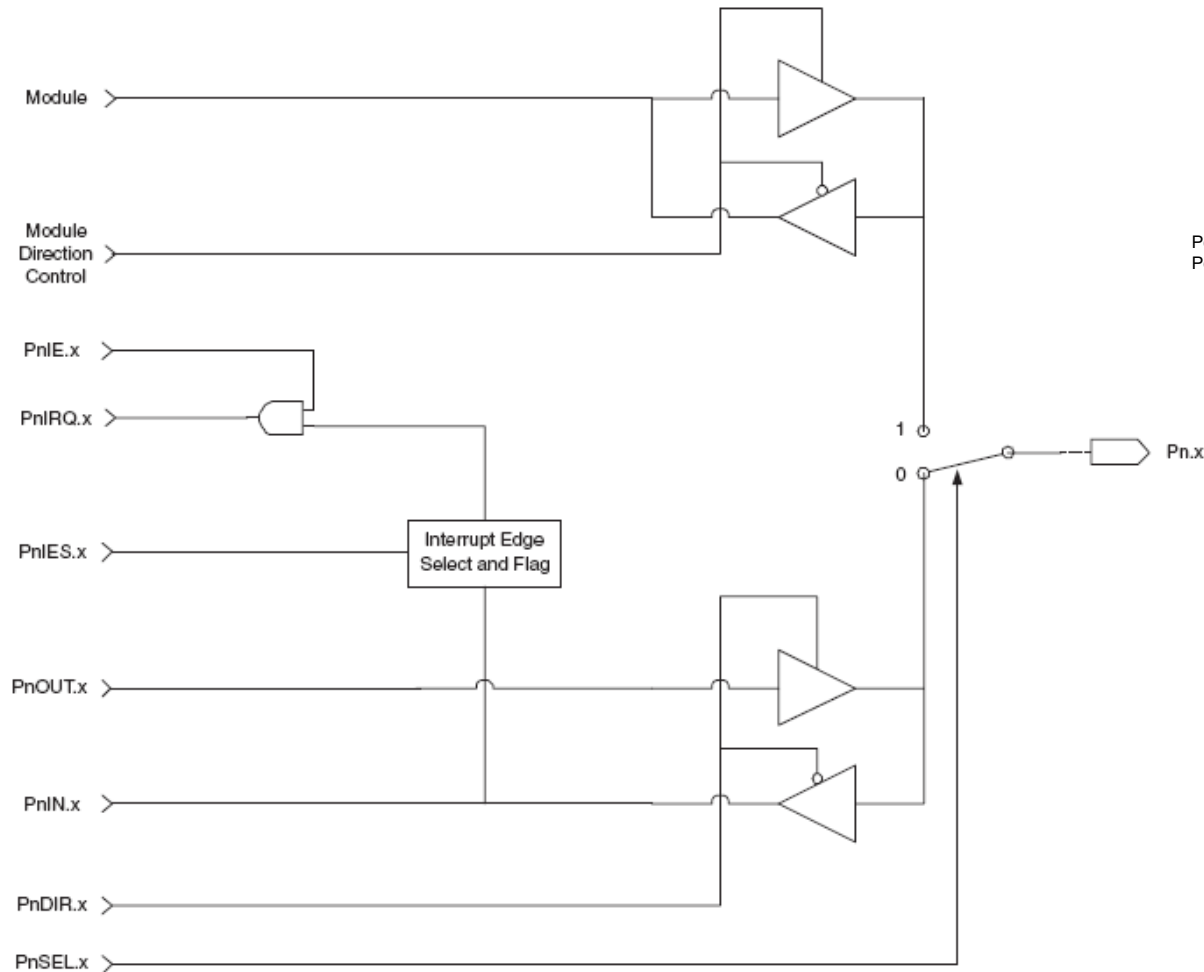


Bild: Interruptable I/O Pin Logic

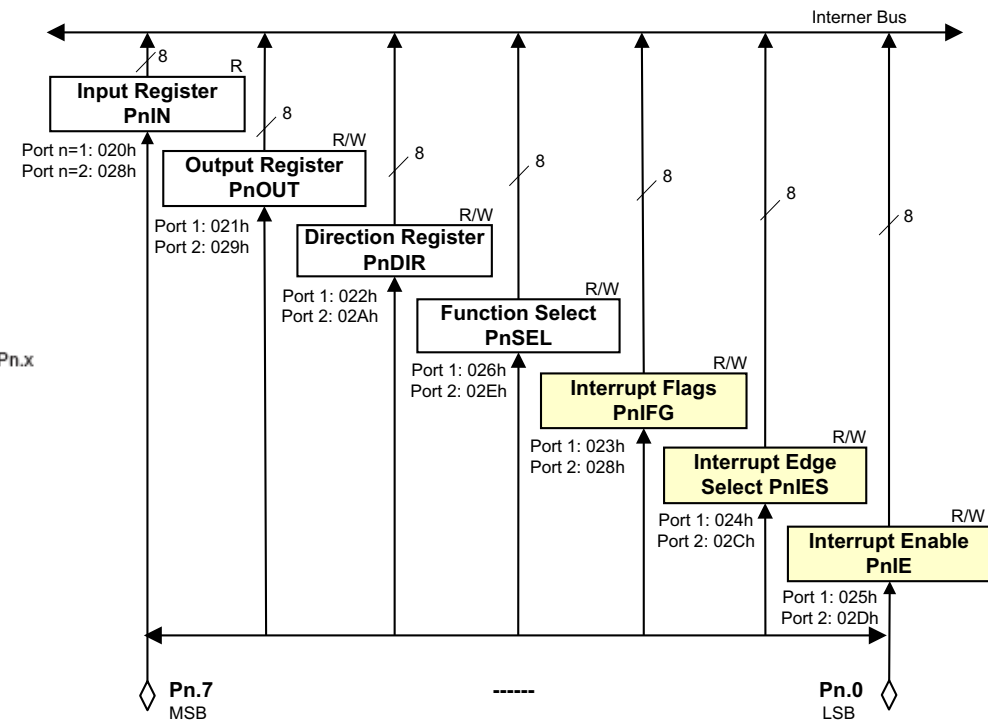


Bild: Digital I/O-Ports mit Interruptfähigkeit (Ports 1 u. 2)

6. Der digitale Ein- und Ausgabeport P1

Konfiguration der digitalen Input/Output Portschnittstelle P1

- Steuerregister P1SEL, P1DIR, P1IE, P1IES
- Datenregister P1OUT, P1IN
- Statusregister P1IFG

Port	Register	Short Form	Address	Register Type	Initial State
P1	Input	P1IN	020h	Read only	–
	Output	P1OUT	021h	Read/write	Unchanged
	Direction	P1DIR	022h	Read/write	Reset with PUC
	Interrupt Flag	P1IFG	023h	Read/write	Reset with PUC
	Interrupt Edge Select	P1IES	024h	Read/write	Unchanged
	Interrupt Enable	P1IE	025h	Read/write	Reset with PUC
	Port Select	P1SEL	026h	Read/write	Reset with PUC
	Resistor Enable	P1REN	027h	Read/write	Reset with PUC

Bild: Digital I/O Registers P1

6. Der digitale Ein- und Ausgabeport P1

Konfiguration der digitalen Input/Output Portschnittstelle P1

Function Select Registers PnSEL -> Steuerregister P1SEL, P2SEL

Die P1- und P2-Portpins werden oft mit anderen Peripheriemodulen gemultiplext, um die Gesamtzahl der Pins bei MSP430-Devices zu reduzieren, s. a. MSP430 Datenblatt.

Die Steuerregister P1SEL und P2SEL werden verwendet, um die gewünschte Pin-Funktion, z. B. digitaler I/O-Port oder ein anderes Peripheriemodul, auszuwählen. Jedes Register enthält acht Bits, die jedem Pin des Hardwareports entsprechen. Die Funktion jedes Pins ist individuell wählbar.

Alle Bits in diesen Registern werden durch das Power Up Clear (PUC)-Signal zurückgesetzt. Die Bit-Definitionen sind wie folgt:

Bit = 0: Das Pin x von Port P1 / P2 arbeitet als digitales I/O

Bit = 1: Das Pin x von Port P1 / P2 ist einem anderen Peripheriemodul zugeordnet

6. Der digitale Ein- und Ausgabeport P1

Konfiguration der digitalen Input/Output Portschnittstelle P1

Direction Registers PnDIR -> Steuerregister P1DIR, P2DIR

Die Richtungsregister enthalten acht unabhängige Bits, die die Richtung des I/O-Pins festlegen. Alle Bits werden durch das PUC-Signal zurückgesetzt. Wenn

Bit = 0: Der Portpin ist auf Eingangsrichtung geschaltet (3-State)

Bit = 1: Der Portpin ist in Ausgangsrichtung geschaltet

6. Der digitale Ein- und Ausgabeport P1

Konfiguration der digitalen Input/Output Portschnittstelle P1

Input_Registers P1IN, P2IN -> Dateneingabe

Beide Input-Register sind Nur-Lese-Register, die die Signale an den I/O-Pins wiedergeben.

Output Registers P1OUT, P2OUT -> Datenausgabe

Jedes Ausgangsregister zeigt die Informationen des Ausgangspuffers an. Der Ausgangspuffer kann von allen Befehlen, die auf ein Ziel schreiben, verändert werden.

Wird das Ausgangsregister zurückgelesen, so ist der Inhalt des Ausgangspuffers unabhängig von der Pin-Richtung. Ein Richtungswechsel verändert den Inhalt des Ausgangspuffers nicht.

6. Der digitale Ein- und Ausgabeport P1

Konfiguration der digitalen Input/Output Portschnittstelle P1

Interrupt Enable P1IE, P2IE

Jedes Interrupt-Freigabe-Register (interrupt enable register) enthält Bits zur Freigabe des Interrupt-Flags für jeden E/A-Pin des Ports. Jedes der sechzehn Bits, die den Pins P1.0 bis P1.7 und P2.0 bis P2.7 entsprechen, befindet sich in den Registern P1IE und P2IE. Wenn

Bit = 0: Die Interrupt-Anforderung ist gesperrt

Bit = 1: Die Interruptanforderung ist aktiviert

Hinweis: Nur Port P1, Port P2 sind interruptfähig. Nur Übergänge, nicht statische Pegel, lösen Interrupts aus.

Ist bei der Ausführung des RETI-Befehls noch ein Interrupt-Flag gesetzt, z. B. bei einer Transition während der Interrupt-Service-Routine, kommt es nach Abschluss von RETI erneut zu einem Interrupt. Damit ist sichergestellt, dass jeder Übergang von der Software quittiert wird.

6. Der digitale Ein- und Ausgabeport P1

Konfiguration der digitalen Input/Output Portschnittstelle P1

Interrupt-Edge Select P1IES

Dieses Lese-/Schreibregister wählt den Übergang aus, bei dem ein Interrupt ausgelöst werden soll.

$PnIES.x = 1$: Wenn es **gesetzt** ist, tritt ein **Interrupt bei einem High-Low-Übergang** an dem entsprechenden Pin auf.

$PnIES.x = 0$: Wenn es **gelöscht ist**, tritt ein **Interrupt bei einem Low-to-High-Übergang** an dem entsprechenden Pin auf.

6. Der digitale Ein- und Ausgabeport P1

Konfiguration der digitalen Input/Output Portschnittstelle P1

Interrupt Flag Register P1IFG, P2IFG

Jedes PnIFGx-Bit ist dem Interrupt-Flag für seinen entsprechenden I/O-Pin zugeordnet.

Das entsprechende PnIFGx-Bit in diesem Lese-/Schreibregister wird automatisch gesetzt, wenn ein Interrupt erzeugt wird. D.h. das PnIFGx-Bit wird gesetzt, wenn die ausgewählte Eingangssignalfanke (High->Low oder Low->High) an dem Pin auftritt.

Alle PnIFGx-Interrupt-Flags lösen einen Interrupt aus, wenn ihr entsprechendes PnIE-Bit und das Global Interrupt Enable (GIE)-Bit gesetzt sind. Dieses Register kann manuell beschrieben werden und erzeugt so per Software einen Interrupt, wenn ein High-Pegel geschrieben wird.

Jedes PnIFG-Flag muss per Software zurückgesetzt werden. Wenn ein Interrupt auftritt, muss dieses Flag vor dem RETI-Befehl gelöscht werden, sonst ruft derselbe Interrupt die ISR ein zweites Mal auf.

Hinweise:

- Jedes PnIFG-Flag kann auch per Software gesetzt werden, so dass ein softwareinitiiertes Interrupt erzeugt werden kann (Softwareinterrupt).

Bit = 0: Es steht kein Interrupt an

Bit = 1: Es steht ein Interrupt an

- Nur Übergänge, nicht statische Pegel, verursachen Interrupts.

6. Der digitale Ein- und Ausgabeport P1

LED Blinken - Programmbeispiel

// Das Programm lässt die LED über Portpin 1.3 fortwährend blinken

// Dateiname: LEDblinken1.c Bitoperationen XOR ^ OR | AND &

#include <msp430x44x.h>

void main(**void**)

{

volatile unsigned int i = 50000; // if not volatile, the delay loop

WDTCTL = WDTPW + WDTCTL; // stop watchdog timer

// Konfiguration der Funktion des PortPins 1.3 -> soll als GPIO arbeiten

P1SEL = P1SEL | 0x08; // Set Portpin 1.3 as digital I/O port

// Konfiguration der Wirkrichtung des PortPins 1.3 -> soll als Ausgabepin arbeiten

P1DIR = P1DIR | 0x08; // Set Pin P1.3 to output direction

// Konfiguration des Interruptfähigkeit -> Port 1 Interrupts abschalten

P1IE = P1IE & ~0x08; // Interrupt für PortPin P1.3 deaktivieren

while(1) // Endloschleife

{

P1OUT = P1OUT ^ 0x08; // Toggle P1.3 using exclusive-OR

i = 50000;

while (i-- != 0); // Delay

}

} // end main()

Inhaltsverzeichnis

1. Download der Praktikumsunterlagen / Veranstaltungsort
2. Ziele und Aufgabenbeschreibung des Versuchs V1
3. Rekapitulation Binäre Operatoren
4. Überblick Mikrocontroller TI MSP430 EVAL Board
5. Der digitale Ein- und Ausgabeport P1
- 6. Übung LED Blinken**
7. Die Low Power Modi des Mikrocontrollers TI MSP430

7. Übung 1: LED Blinken

Ein LOW-Pegel an der Kathode der LED schaltet die LED ein, ein High-Pegel schaltet die LED aus.
Die LED ist über Portpin 1.3 der MCU TI MSP430F449 angeschlossen.

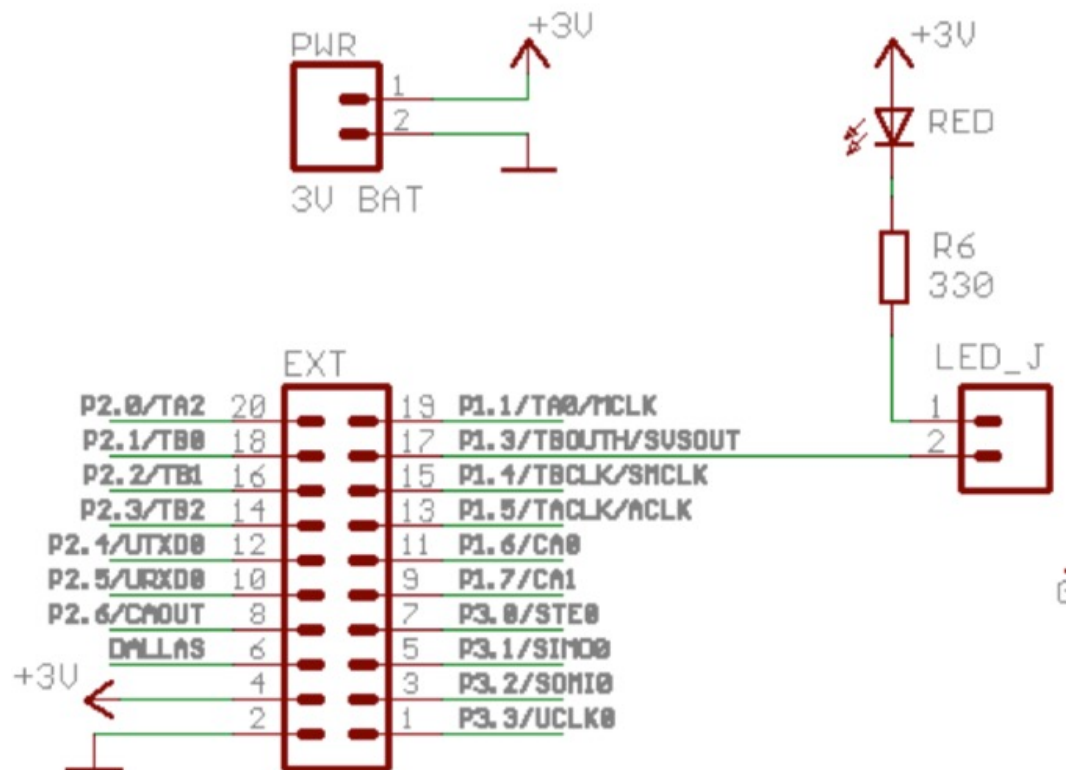
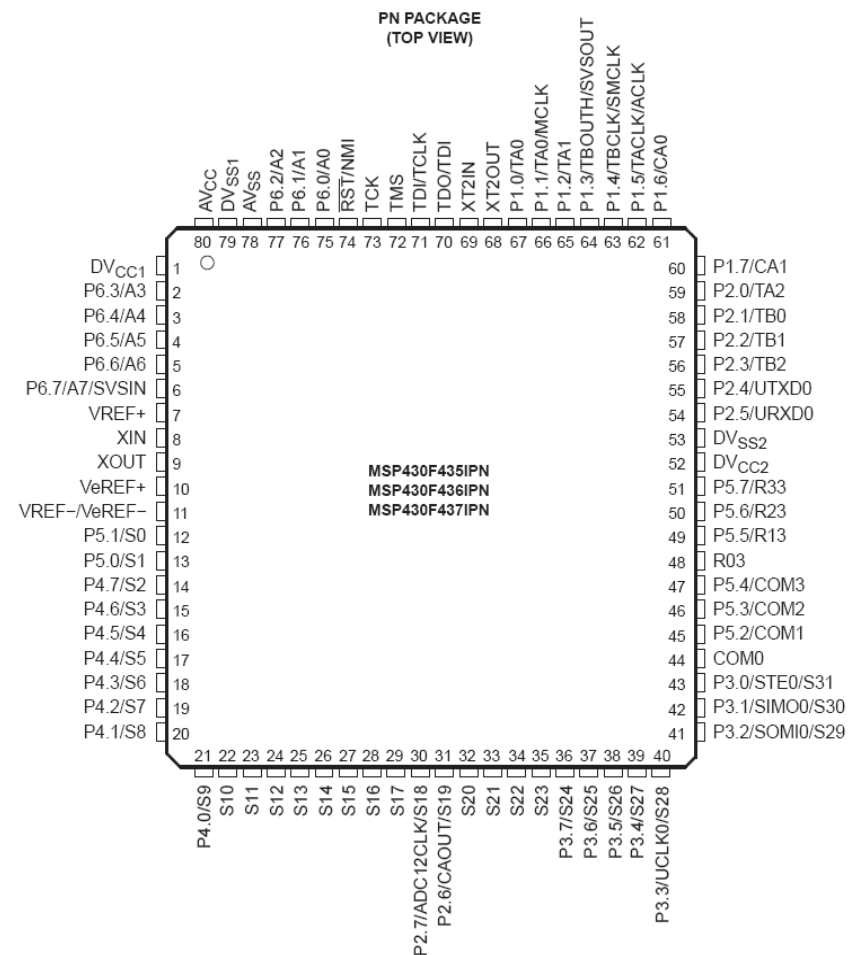


Bild: Ausschnitt Pinbelegung Port 1

Mehrfachbelegung der I/O-Pins



TERMINAL		I/O	DESCRIPTION
PN NAME	NO.		
P1.6/CA0	81	I/O	General-purpose digital I/O / Comparator_A input
P1.5/TACLK/ ACLK	82	I/O	General-purpose digital I/O / Timer_A, clock signal TACLK input / ACLK output (divided by 1, 2, 4, or 8)
P1.4/TBCLK/ SMCLK	83	I/O	General-purpose digital I/O / input clock TBCLK—Timer_B7 / submain system clock SMCLK output
P1.3/TBOutH/ SVSOut	84	I/O	General-purpose digital I/O / switch all PWM digital output ports to high impedance—Timer_B7 TB0 to TB6 / SVS: output of SVS comparator
P1.2/TA1	85	I/O	General-purpose digital I/O / Timer_A, Capture: CCI1A, compare: Out1 output
P1.1/TA0/MCLK	86	I/O	General-purpose digital I/O / Timer_A, Capture: CCI0B / MCLK output. Note: TA0 is only an input on this pin.
P1.0/TA0	87	I/O	General-purpose digital I/O / Timer_A, Capture: CCI0A input, compare: Out0 output
XT2OUT	88	O	Output terminal of crystal oscillator XT2
XT2IN	89	I	Input port for crystal oscillator XT2. Only standard crystals can be connected.
TDO/TDI	90	I/O	Test data output port. TDO/TDI data output or programming data input terminal
TDI	91	I	Test data input. TDI is used as a data input port. The device protection fuse is connected to TDI.
TMS	92	I	Test mode select. TMS is used as an input port for device programming and test.
TCK	93	I	Test clock. TCK is the clock input port for device programming and test.
RST/NMI	94	I	Reset input or nonmaskable interrupt input port
P6.0/A0	95	I/O	General-purpose digital I/O, analog input a0—12-bit ADC
P6.1/A1	96	I/O	General-purpose digital I/O, analog input a1—12-bit ADC
P6.2/A2	97	I/O	General-purpose digital I/O, analog input a2—12-bit ADC
AVSS	98		Negative terminal that supplies SVS, brownout, oscillator, FLL+, comparator_A, port 1, and LCD resistive divider circuitry.
DVSS1	99		Digital supply voltage, negative terminal. Supplies all digital parts, except those which are supplied via AVCC/AVSS.
AVCC	100		Positive terminal that supplies SVS, brownout, oscillator, FLL+, comparator_A, port 1, and LCD resistive divider circuitry; must not power up prior to DVCC1/DVCC2.

7. Übung 2: Taste B1 einlesen und LED toggeln (1)

An B1 liegt standardmäßig durch Pull-Up Widerstand High-Pegel.

Bei Tastenbetätigung liegt am Signal B1 und somit am Portpin P3.4 ein Low-Pegel an.

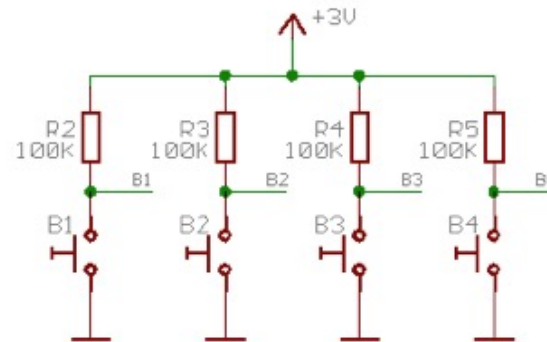
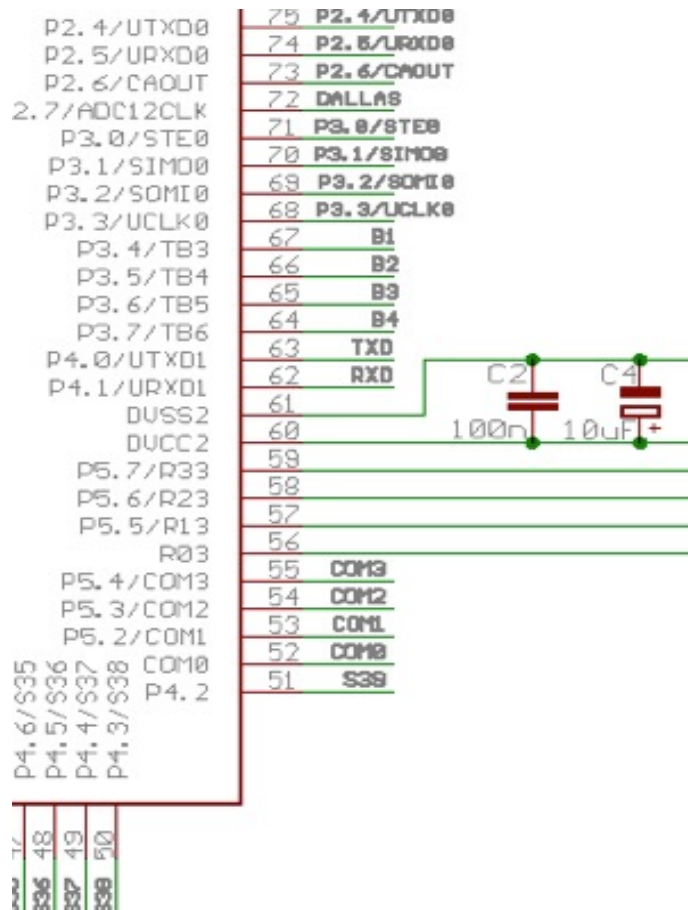


Bild: Ausschnitt Pinbelegung Port 1

7. Übung 2: Taste B1 einlesen und LED toggeln (2)

// Das Programm liest den Tastenpegel B1 ein toggelt, wenn Taste betätigt wird

// Dateiname: LEDtoggle1.c Bitoperationen XOR ^ OR | AND &

```
#include <msp430x44x.h>
```

```
void main(void)
```

```
{
    volatile unsigned int i = 50000;
    WDTCTL = WDTPW + WDTHOLD;    // Stop watchdog timer

    P1SEL = P1SEL & ~0x08 ;        // Set P1.3 as digital I/O Portpin
    // Set P3.4 as digital I/O (GPIO)

    P1DIR = P1DIR | 0x08 ;        // Set PortPin P1.3 to output direction
    P1OUT = P1OUT | 0x08;        // LED ausschalten (P1.3 = High)
    // Set P3.4 to input direction

    P1IE = P1IE & ~0x08 ;        // Interrupt fuer PortPin P1.3 deaktivieren

    while(1)    // Endloschleife
    {

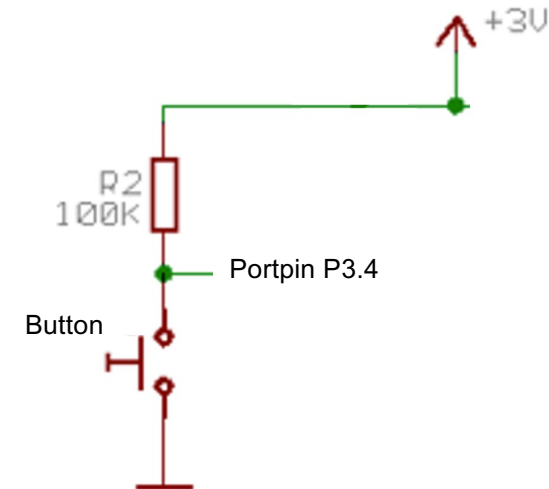
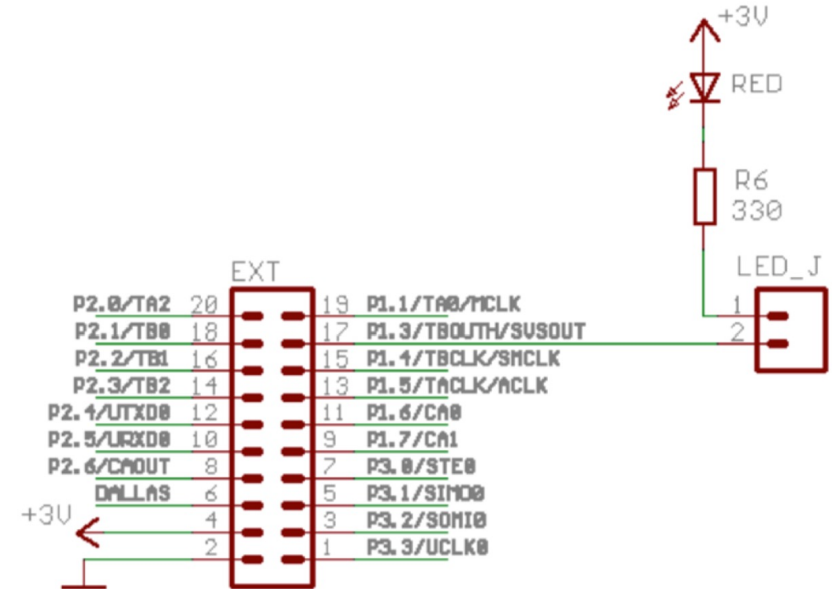
        // Wenn Portpin P3.4 == LOW, dann LED toggeln

        // LED einschalten (P3.4)

        // LED ausschalten (P3.4)

        while (i-- != 0);
        i = 50000;

    }
} // end main()
```



Inhaltsverzeichnis

1. Download der Praktikumsunterlagen / Veranstaltungsort
2. Ziele und Aufgabenbeschreibung des Versuchs V1
3. Rekapitulation Binäre Operatoren
4. Überblick Mikrocontroller TI MSP430 EVAL Board
5. Die serielle Schnittstelle
6. Der digitale Ein- und Ausgabeport P1
7. Übung LED Blinken
8. **Low Power Modi des Mikrocontrollers TI MSP430**

Low Power Betriebsarten und ISR-Programmierung (1)

Protokollierung in Abhängigkeit der Betriebsart Polling versus Interruptbetrieb

Fall	Betriebsart	I [μ A]
1	Programm im Polling-Betrieb , mit CPU Active mode , LED Ein (Versuch1.c)	
2	Programm im Polling-Betrieb , mit CPU Active mode , LED Aus (Versuch1.c)	
3	Programm im Interrupt-Betrieb , mit CPU Active mode , LED Ein (Versuch2.c)	
4	Programm im Interrupt-Betrieb , mit CPU Active mode , LED Aus (Versuch2.c)	
5	Programm im Interrupt-Betrieb , mit Low Power Mode (LPM3) , LED Ein (Versuch3.c)	
6	Programm im Interrupt-Betrieb , mit Low Power Mode (LPM3) , LED Aus (Versuch3.c)	
7	Programm im Interrupt-Betrieb , mit Low Power Mode (LPM4) , LED Aus (Versuch3.c)	

Tabelle 1: Stromaufnahme in Abhängigkeit der Betriebsart

Low Power Betriebsarten und ISR-Programmierung (2)

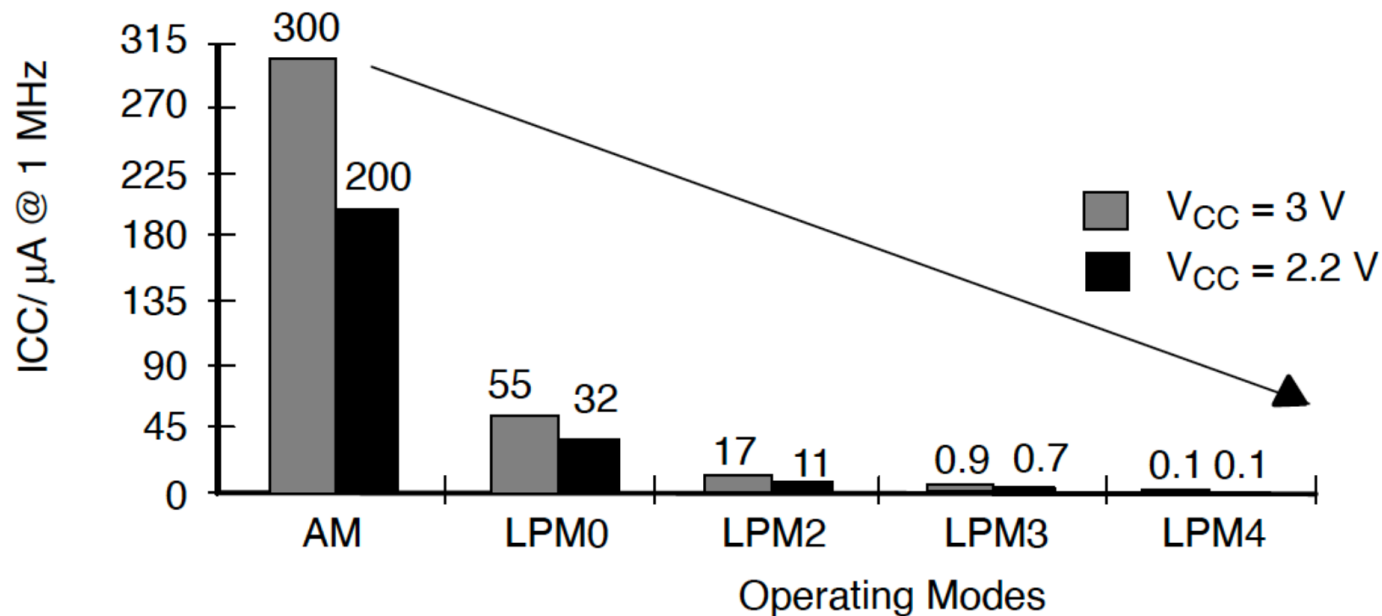
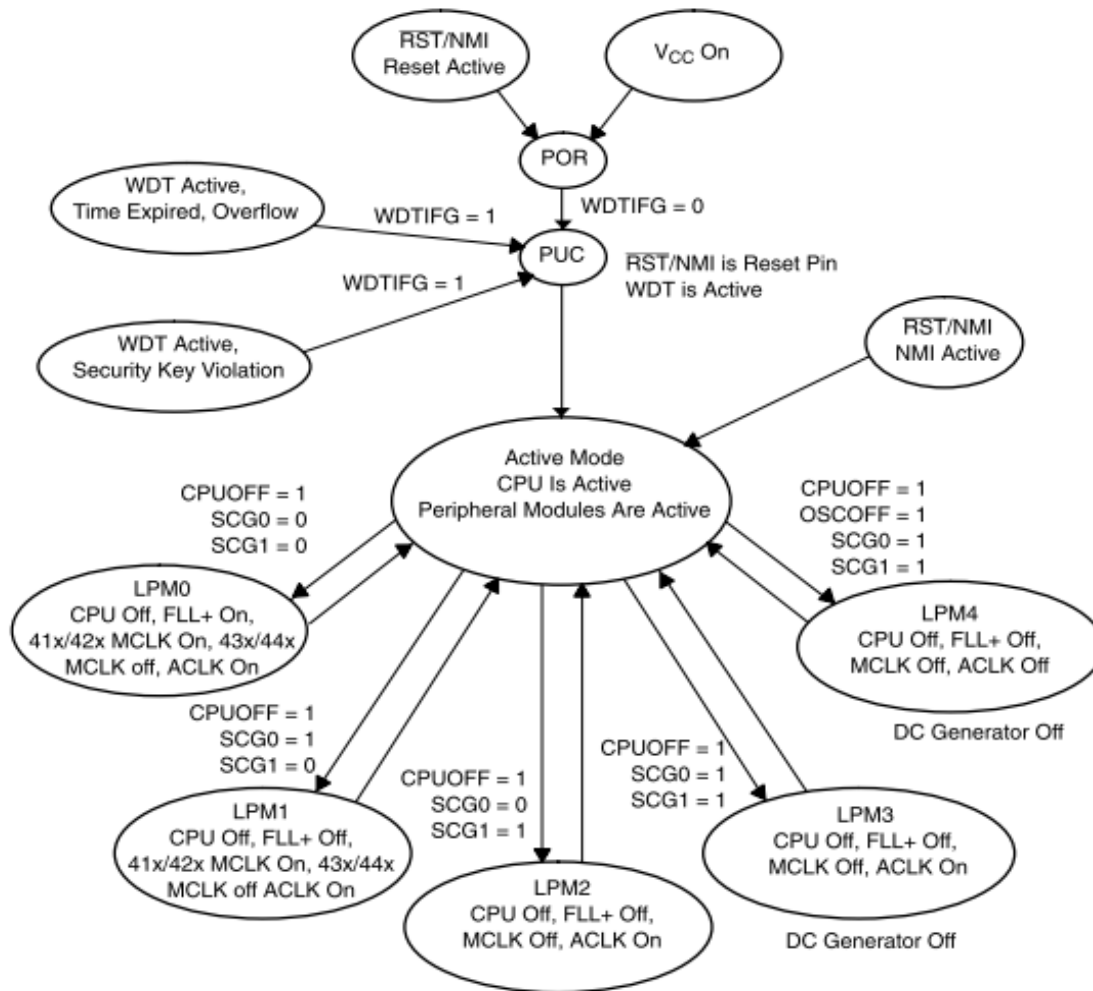


Bild: Typischer Stromverbrauch von MSP430F44x MCUs in Abhängigkeit der Betriebsart

Low Power Betriebsarten und ISR-Programmierung (3)



Die Betriebsart des Prozessors wird im Prozessor Status Word festgelegt.

Hier werden u. a. die Low Power Betriebsarten LPM0 - LPM4 mittels der Status Register Bits SCG1, SCG0, OSCOFF und CPUOFF bestimmt

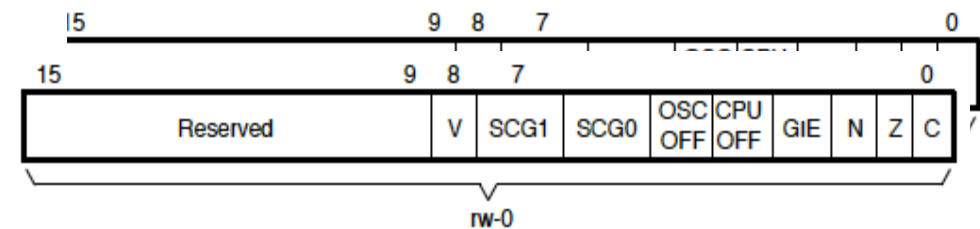


Bild: Aufbau des Prozessor Status Registers (Prozessor Statuswort, PSW)

Bild 12: MSP430 Operating Modes

Low Power Betriebsarten und ISR-Programmierung (4)

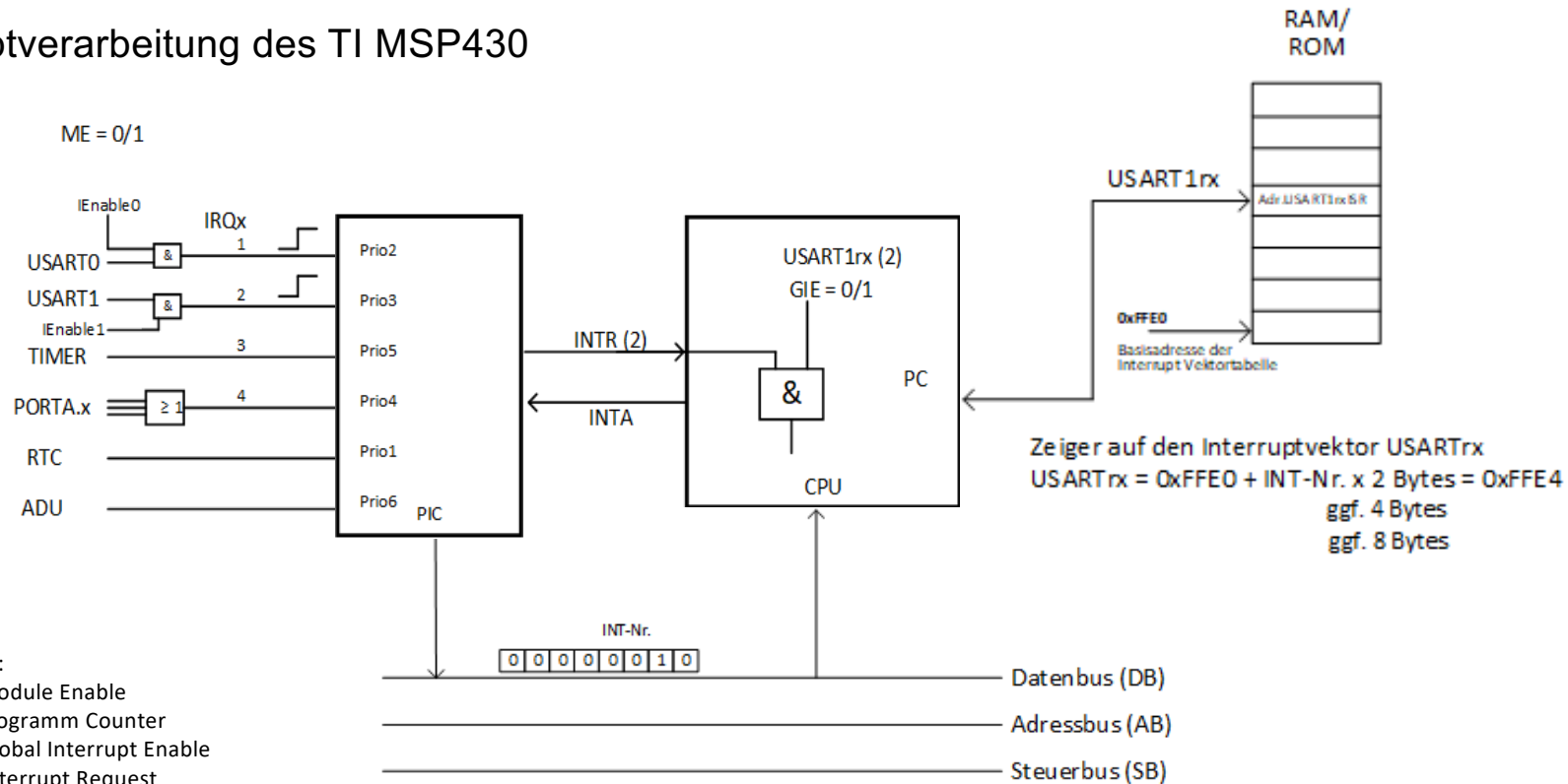
Bit	Description
Reserved	Reserved
V	Overflow bit. This bit is set when the result of an arithmetic operation overflows the signed-variable range. ADD(.B), ADDX(.B,.A), ADDC(.B), ADDCX(.B.A), ADDA Set when: positive + positive = negative negative + negative = positive otherwise reset SUB(.B), SUBX(.B,.A), SUBC(.B), SUBCX(.B,.A), SUBA, CMP(.B), CMPX(.B,.A), CMPA Set when: positive - negative = negative negative - positive = positive otherwise reset
SCG1	System clock generator 1. This bit, when set, turns off the DCO dc generator if DCOCLK is not used for MCLK or SMCLK.
SCG0	System clock generator 0. This bit, when set, turns off the FLL+ loop control.
OSCOFF	Oscillator Off. This bit, when set, turns off the LFXT1 crystal oscillator when LFXT1CLK is not used for MCLK or SMCLK.
CPUOFF	CPU off. This bit, when set, turns off the CPU.
GIE	General interrupt enable. This bit, when set, enables maskable interrupts. When reset, all maskable interrupts are disabled.
N	Negative bit. This bit is set when the result of an operation is negative and cleared when the result is positive.
Z	Zero bit. This bit is set when the result of an operation is zero and cleared when the result is not zero.
C	Carry bit. This bit is set when the result of an operation produced a carry and cleared when no carry occurred.

SCG1	SCG0	OSCOFF	CPUOFF	Mode	CPU and Clocks Status
0	0	0	0	Active	CPU is active, all enabled clocks are active
0	0	0	1	LPM0	CPU, MCLK are disabled (41x/42x peripheral MCLK remains on) SMCLK, ACLK are active
0	1	0	1	LPM1	CPU, MCLK, DCO osc. are disabled (41x/42x peripheral MCLK remains on) DC generator is disabled if the DCO is not used for MCLK or SMCLK in active mode SMCLK, ACLK are active
1	0	0	1	LPM2	CPU, MCLK, SMCLK, DCO osc. are disabled DC generator remains enabled ACLK is active
1	1	0	1	LPM3	CPU, MCLK, SMCLK, DCO osc. are disabled DC generator disabled ACLK is active
1	1	1	1	LPM4	CPU and all clocks disabled

Bild 13: LPM-relevante Bits im Prozessor Statuswort (PSW)

Low Power Betriebsarten und ISR-Programmierung (5)

Interruptverarbeitung des TI MSP430



Legende:

ME - Module Enable

PC - Programm Counter

GIE - Global Interrupt Enable

INTR - Interrupt Request

INTA - Interrupt Acknowledge

PIC - Programmable Interrupt Controller

IRQx - Interrupt Request

ADU - Analog Digital Umsetzer

RTC - Real-time clock

USART1rx - Universal Asynchronous Receiver Transmitter

- Receive Interrupt USART1

Low Power Betriebsarten und ISR-Programmierung (6)

```
// entnommen aus msp430x44x.h
/*****
 * Interrupt Vectors (offset from 0xFFE0)
 *****/

#define BASICTIMER_VECTOR    (0 * 2u) /* 0xFFE0 Basic Timer */
#define PORT2_VECTOR         (1 * 2u) /* 0xFFE2 Port 2 */
#define USART1TX_VECTOR      (2 * 2u) /* 0xFFE4 USART 1 Transmit */
#define USART1RX_VECTOR      (3 * 2u) /* 0xFFE6 USART 1 Receive */
#define PORT1_VECTOR         (4 * 2u) /* 0xFFE8 Port 1 */
#define TIMERA1_VECTOR        (5 * 2u) /* 0xFFEA Timer A CC1-2, TA */
#define TIMERA0_VECTOR        (6 * 2u) /* 0xFFEC Timer A CC0 */
#define ADC_VECTOR           (7 * 2u) /* 0xFFEE ADC */
#define USART0TX_VECTOR      (8 * 2u) /* 0xFFFF0 USART 0 Transmit */
#define USART0RX_VECTOR      (9 * 2u) /* 0xFFFF2 USART 0 Receive */
#define WDT_VECTOR           (10 * 2u) /* 0xFFFF4 Watchdog Timer */
#define COMPARATORA_VECTOR   (11 * 2u) /* 0xFFFF6 Comparator A */
#define TIMERB1_VECTOR        (12 * 2u) /* 0xFFFF8 Timer B CC1-6, TB */
#define TIMERB0_VECTOR        (13 * 2u) /* 0xFFFFA Timer B CC0 */
#define NMI_VECTOR           (14 * 2u) /* 0xFFFFC Non-maskable */
#define RESET_VECTOR         (15 * 2u) /* 0xFFFFE Reset [Highest
Priority] */
#define UART1TX_VECTOR        USART1TX_VECTOR
#define UART1RX_VECTOR        USART1RX_VECTOR
#define UART0TX_VECTOR        USART0TX_VECTOR
#define UART0RX_VECTOR        USART0RX_VECTOR
/*****/
```

Interruptvektoren des TI MSP430x449

Low Power Betriebsarten und ISR-Programmierung (7)

Programmbeispiel

Das Programm wartet im Low-Power-Mode auf den Empfang eines Zeichens über die Serielle-Schnittstelle und sendet jedes empfangene Zeichen zurück.

```
#include <msp430x44x.h>

void init_uart(void)      // Initialisierung der UART V24 Schnittstelle
{
    // Baudrate einstellen
    U1BR0= 0x03;          // Baudraten Kontrollregister 0
    U1BR1= 0x00;          // Baudraten Kontrollregister 1
    U1MCTL= 0x4A;         // Modulationskontrollregister

    //Einstellen der Clocksource fuer den Baudratengenerator
    U1TCTL= 0x10;         // Source select SSEL0 = 0x10 = ACLK; Source select SSEL1=SMCLK;

    // Einstellen der Schnittstellenparameter
    U1CTL=0xD0;           // 9600, Parity Enable, Even Parity, 8 Data Bits , 1 Stopbit

    // Freigabe der Schnittstelle
    ME2 |= 0x30;          // Transmit und receive enable
    IE2 |= URXIE1;        // Enable USART0 RX interrupt

    //Einstellen des Ports
    P4SEL |= 0x03;        // Bit 0 und 1 == 1 -> Peripheral module function
    P4DIR |= 0x01;        // Bit 0 Output (Transmit)
}
```

```
void main( void )
{
    WDTCTL = WDTPW + WDTHOLD;    // Stop watchdog timer to prevent timeout reset

    init_uart();                 // Initialisierung der Schnittstelle

    _EINT();                     // Allgemeine Interruptfreigabe (GIE=1)

    while(1)                    // Endloswiederholung
    {
        _BIS_SR(LPM3_bits + GIE);    // enter LPM3, program stops

        while( (IFG2 & UTXIFG1)==0); // Ist Transmitter frei für neue Daten?
        TXBUF1=RXBUF1;              // Zeichen einlesen und zurück senden
    }

#pragma vector = UART1RX_VECTOR
__interrupt void usart1_rx (void) // ISR usart1_rx fuer seriellen Empfang
{
    _BIC_SR_IRQ(LPM3_bits);        // modify PSW auf Stack

    // RETI - Return from Interrupt
}
```

Programm-Listing 3: Hauptprogramm Versuch1-3.c

Fine.